

Solve your problems with PENLAB

Jan Fiala, Numerical Algorithms Group

Michal Kočvara, University of Birmingham

Michael Stingl, University of Erlangen

software package for nonlinear (and) semidefinite optimization

It is

free (under GNU GPL)
open source
fully coded in Matlab

It uses

verified algorithms from
PENSDP, PENBMI and PENNON¹

Pre-programmed interfaces for

linear and bilinear SDP
nonlinear optimization with AMPL input

Other features

YALMIP compatibility for LMI and BMI
SeDuMi and SDPA input

It solves

nonlinear continuous optimization problems
linear matrix inequalities
bilinear matrix inequalities
polynomial matrix inequalities
nonlinear semidefinite programming problems

Pre-programmed applications

nearest correlation matrix
Lyapunov stability
static output feedback
truss topology optimization

¹ **PENSDP, PENBMI, PENNON** are codes for linear, bilinear and nonlinear SDP developed by PENOPT GbR, www.penopt.com

Available as binaries for Linux and MS Windows
Free academic license

PENNON will be included in the next release of NAG numerical libraries

www.nag.co.uk/numeric/numerical_libraries.asp

The problem

$$\begin{aligned} \min_{x, Y_1, \dots, Y_k} \quad & f(x, Y) \quad [x \in \mathbb{R}^n, Y \in \mathbb{R}^{n \times n} \text{ symmetric}] \\ \text{s.t.} \quad & g_i(x, Y) \leq 0, \quad i = 1, \dots, m_g \\ & h_i(x, Y) = 0, \quad i = 1, \dots, m_h \\ & A_i(x, Y) \prec 0, \quad i = 1, \dots, m_A \\ & \underline{\lambda}_i \prec Y_i \prec \bar{\lambda}_i, \quad i = 1, \dots, k \end{aligned}$$

Specific examples

Nonlinear optimization

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m_g \\ & h_i(x) = 0, \quad i = 1, \dots, m_h \end{aligned}$$

Bilinear matrix inequalities

$$\begin{aligned} \min_x \quad & \frac{1}{2} x^T A x + c^T x \\ \text{s.t.} \quad & Q_0^i + \sum_{k=1}^n x_k Q_k^i + \sum_{k=1}^n \sum_{l=1}^n x_k x_l Q_{kl}^i \succ 0, \quad i = 1, \dots, m \end{aligned}$$

Nearest correlation matrix with constrained κ

$$\begin{aligned} \min_{X, z} \quad & \sum_{i,j} \left(\frac{1}{z} X_{ij} - H_{ij} \right)^2 \\ \text{s.t.} \quad & X_{ii} - z = 0, \quad i = 1, \dots, n \\ & I \prec X \prec \kappa I \end{aligned}$$

Lyapunov stability

$$\begin{aligned} \min_P \quad & \text{tr } P \\ \text{s.t.} \quad & A^T P + P A \prec 0, \quad P \succ 0 \\ \min_P \quad & \text{tr } P \\ \text{s.t.} \quad & A^T P A - P \prec 0, \quad P \succ 0 \end{aligned}$$

What next

nonlinear SOCP
automatic differentiation
more pre-programmed interfaces
iterative methods
speed improvements

The code

free open-source MATLAB implementation of PENNON
attention to clarity of the code rather than performance
easy to extend and further develop
suitable for classrooms

Usage

The user needs to prepare a MATLAB structure which describes the problem parameters, such as number of variables, number of constraints, lower and upper bounds, etc.

For a generic problem, the user must define callback

Functions for function value, gradient, and Hessian of the objective function, NLP constraints and matrix constraint.

Pre-programmed interfaces

PENLAB contains several pre-programmed interfaces for standard optimization problems with standard inputs. For these problems, the user does not have to create the `penm` object, nor the callback functions.

Example, generic interface

$$\begin{aligned} \min_{x_1, x_2} \quad & \frac{1}{2} (x_1^2 + x_2^2) \\ \text{s.t.} \quad & \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} x_1 + \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} x_2 + \begin{pmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \succ 0 \end{aligned}$$

```
penm = ex_define;
prob = penlab(penm);
prob.solve(); x = prob.x;
```

```
function [penm] = ex_define()
B = sparse([1 -1 0; -1 1 0; 0 0 1]);
A{1} = sparse([0 1 0; 1 0 0; 0 0 0]);
A{2} = sparse([0 0 0; 0 0 1; 0 1 0]);
penm = []; penm.Nx=2; penm.NALIN=1;
penm.lbA=zeros(1,1);
penm.objfun = @(x,Y) (-.5*(x(1)^2+x(2)^2));
penm.objgrad = @(x,Y) (-[x(1); x(2)]);
penm.objhess = @(x,Y) (-eye(2,2));
penm.mconfun = @(x,Y,k) (B+A{1}*x(1)+A{2}*x(2));
penm.mcongrad = @(x,Y,k,i) (A{i});
end
```

Examples of pre-programmed interfaces

```
%YALMIP commands
A = [-1 2;-3 -4]; B=-[1;1];
P = sdpvar(2,2); K = sdpvar(1,2);
F = [(A+B*K)'*P+P*(A+B*K) < -eye(2); P>eye(2)]
yalpen = export(F,trace(P),...
    sdpsettings('solver','penbmi'),[],[],1);
%PENLAB commands
bmi = yalmip2bmi(yalpen);
penm = bmi_define(bmi);
prob = penlab(penm);
solve(prob); K = (prob.x(4:5))';
```

%Linear semidefinite programming with SDPA input

```
sdpdata = readsdp('datafiles/arch0.dat-s');
penm = sdp_define(sdpdata);
prob = penlab(penm);
prob.solve(); x = prob.x;
```

%Nonlinear optimization with AMPL input

```
penm = nlp_define('datafiles/chain100.n1');
prob = penlab(penm);
prob.solve();
```

PENNON vs PENLAB, COPS3 examples (NLP)							
problem	vars	cons	type	PENNON		PENLAB	
				CPU	Nwt iter	CPU	Nwt iter
elec	600	200	=	40	224	31	135
pinene	8000	7995	=	1	7	11	17
Dirichlet	4491	21	<	151	71	73	68
henon	2701	21	<	57	128	63	158
minsurf	5000	5000	box	1	20	97	203
duct	2895	8601	= & <	6	19	9	11
marine	6415	6392	< & box	2	39	22	35

PENSDP vs PENLAB, SDPLIB + TOPO examples

problem	vars	matrix	constr.	PENNON		PENLAB	
	number	size		CPU	Nwt iter	CPU	Nwt iter
control3	136	2	30	1	103	20	315
maxG11	800	1	1600	18	41	186	61
qpG11	800	1	1600	43	43	602	64
ss30	132	1	294	20	112	17	63
theta3	1106	1	150	11	52	61	48
buck2	144	2	97	2	74	22	184
vibra2	144	2	97	2	132	35	304
shmup	200	2	441	65	99	172	179
mater2	423	94	11	2	89	70	179

Download PENLAB from <http://web.mat.bham.ac.uk/kocvara/penlab/>

PENLAB = NLP + LMI + BMI + PMI + SDP